Data Engineering

Atticus Tarleton

2026-06-08

Below is the code block that moves the project root to the right place. Basically, it sets the root to this file, then moves back until it reaches the where the git info is stored to determine the code repository root

```
import os
from pathlib import Path

_project_root = Path.cwd()
while not (_project_root / ".git").exists():
    _project_root = _project_root.parent
    if _project_root == _project_root.parent:
        raise RuntimeError(".git not found - are you inside the repo?")
os.chdir(_project_root)
print(os.getcwd())
```

```
/Users/AGT/Desktop/ORCA/OkemoDataLearningProject
```

Reading in the data

Below is a code block that takes a csv and reads it in as a pandas dataframe. what this means is it takes the csv stored on the hard drive and turns it into a database stored on RAM that is much easier to access and manipulate

```
import pandas as pd
from pathlib import Path

path = Path("Data/raw_data/ahrfsn2025.csv")

raw_pandas_dataframe = pd.read_csv(path) #reading the data
raw_pandas_dataframe.head() #checking to make sure the above line worked
```

	fips_st	st_abbrev	phys_wkforc_23	phys_mal_23	phys_fem_23	phys_lt30_23	phys_30_39_23
0	1	AL	12442	8745	3697	1347.0	2871.0
1	2	AK	2158	1228	930	NaN	969.0
2	4	AZ	18253	11945	6308	1195.0	4341.0
3	5	AR	6493	4291	2202	NaN	2045.0
4	6	CA	115435	66542	48893	6227.0	32051.0

The below code takes the csv, and turns the csv into a sql database in memory using duckdb. it then uses duckdb to select everything from the sql database and turn it into a pandas dataframe to display more easily

```
import duckdb

con = duckdb.connect()
con.execute(f"""
  CREATE OR REPLACE VIEW healthcare AS
  SELECT * FROM read_csv('{path}')
""")
raw_sql_dataframe = con.execute("SELECT * FROM healthcare").df()
raw_sql_dataframe.head()
```

	fips_st	st_abbrev	phys_wkforc_23	phys_mal_23	phys_fem_23	phys_lt30_23	phys_30_39_23
0	01	AL	12442	8745	3697	1347	2871
1	02	AK	2158	1228	930	<NA>	969
2	04	AZ	18253	11945	6308	1195	4341
3	05	AR	6493	4291	2202	<NA>	2045
4	06	CA	115435	66542	48893	6227	32051

Adding in more features

Next block is using pandas to describe the data.

The region of each state is determined by the census, coming from this website: https://www2.census.gov/geo/pdfs/maps-data/maps/reference/us_regdiv.pdf

The regions and their acronym's - New England: NE - Middle Atlantic: MA - East North Central: ENC - West North Central: WNC - South Atlantic: SA - East South Central: ESC - West South Central: WSC - Mountain: M - Pacific: P - Whole United States: US This block will make a list of the column names. It will also print out all the unique state abbreviations, which will total 52, one for each state, the district of columbia, and the United States of

America as a whole. It will also add each state to its census region, with the acroynm's listed above. Finally, a new database will be created with fewer columns, keeping the ones more relevant to the analysis that will be conducted. This means keeping the overall numbers for different professions, but not specifics like the number of new graduates or the number of male or female practitioners.

```
list_col_names = raw_pandas_dataframe.columns.tolist() #this is over 1000 columns, so it won

unique_st_abbrev = raw_pandas_dataframe['st_abbrev'].unique() #all 50 states as well as the U
display(unique_st_abbrev)

# adding a region to each state
raw_pandas_dataframe['region'] = ['ESC', 'P', 'M', 'WSC', 'P', 'M', 'NE', 'SA', 'SA', 'SA',
    'P', 'M', 'ENC', 'ENC', 'WNC', 'WNC', 'ESC', 'WSC', 'NE', 'SA', 'NE', 'ENC', 'WNC', 'ESC
    'WNC', 'M', 'WNC', 'M', 'NE', 'MA', 'M', 'MA', 'SA', 'WNC', 'ENC', 'WSC', 'P', 'MA', 'NE
    'SA', 'WNC', 'ESC', 'WSC', 'M', 'NE', 'SA', 'P', 'SA', 'ENC', 'M', 'US']

# making a smaller database that only contains what I am interested in
list_of_relevant_cols = ['fips_st', 'st_abbrev', 'phys_wkforc_23', 'pa_23', 'rn_23', 'dent_23
    'pharm_23', 'vetin_23', 'podtrst_23', 'chiro_23', 'opto_23', 'psychol_23','conslrs_23',
    'socwk_23', 'pt_23', 'ot_23', 'resp_ther_23', 'spech_path_23', 'massg_ther_23', 'dietn_23
    'medmgr_23', 'med_secrtrs_23', 'clin_lab_techs_23', 'diag_reltd_techs_23', 'emt_parmdcs_2
    'hlth_sup_techs_23', 'med_rec_spec_23', 'med_asst_23', 'pharm_aide_23', 'med_transcrp_23
    'vetin_asst_23', 'phleb_23', 'oth_hlthcar_23', 'pers_care_aide_23', 'nurse_aide_23',
    'popn_pums_23', 'popn_ge60_23', 'popn_24', 'region']
smaller_health_pandas_dataframe = raw_pandas_dataframe[list_of_relevant_cols]
display(smaller_health_pandas_dataframe.head())
```

```
array(['AL', 'AK', 'AZ', 'AR', 'CA', 'CO', 'CT', 'DE', 'DC', 'FL', 'GA',
       'HI', 'ID', 'IL', 'IN', 'IA', 'KS', 'KY', 'LA', 'ME', 'MD', 'MA',
       'MI', 'MN', 'MS', 'MO', 'MT', 'NE', 'NV', 'NH', 'NJ', 'NM', 'NY',
       'NC', 'ND', 'OH', 'OK', 'OR', 'PA', 'RI', 'SC', 'SD', 'TN', 'TX',
       'UT', 'VT', 'VA', 'WA', 'WV', 'WI', 'WY', 'US'], dtype=object)
```

	fips_st	st_abbrev	phys_wkforc_23	pa_23	rn_23	dent_23	pharm_23	vetin_23	podtrst_23
0	1	AL	12442	1301.0	57118	1830.0	5788.0	1503.0	NaN
1	2	AK	2158	NaN	8538	NaN	NaN	NaN	NaN
2	4	AZ	18253	3522.0	67664	4125.0	7795.0	2575.0	374.0
3	5	AR	6493	630.0	34031	920.0	2645.0	496.0	NaN
4	6	CA	115435	13769.0	342017	26082.0	37806.0	10311.0	1272.0

The next block of code is doing the same thing as the above block of code, just in SQL

The results of each sql query will be turned into a dataframe for ease of viewing in python

Notes on differences in difficulty for certain tasks: - Making a new column in SQL was much more difficult - The lack of quotation marks in sql makes it tedious to transfer code from pandas to sql The code chunk below also creates a new table for adding a region and having smaller number of columns. This is because a view is just a snapshot of a table, so to make changes to the dataset requires changing the base table, as the view can't have data added to it.

```
col_names_sql = con.execute("DESCRIBE healthcare").df()
print(col_names_sql)

unique_st_abbrev_sql = con.execute("SELECT DISTINCT st_abbrev FROM healthcare").df()
display(unique_st_abbrev_sql)

state_regions_alphabetical = ['ESC', 'P', 'M', 'WSC', 'P', 'M', 'NE', 'SA', 'SA', 'SA', 'SA'
    'P', 'M', 'ENC', 'ENC', 'WNC', 'WNC', 'ESC', 'WSC', 'NE', 'SA', 'NE', 'ENC', 'WNC', 'ESC
    'WNC', 'M', 'WNC', 'M', 'NE', 'MA', 'M', 'MA', 'SA', 'WNC', 'ENC', 'WSC', 'P', 'MA', 'NE
    'SA', 'WNC', 'ESC', 'WSC', 'M', 'NE', 'SA', 'P', 'SA', 'ENC', 'M', 'US']
fips_code = ['01',
'02','04','05','06','08','09', '10','11', '12', '13', '15','16', '17', '18',
             '19','20','21','22', '23', '24', '25', '26', '27','28', '29',
             '30', '31','32', '33', '34', '35', '36', '37', '38', '39', '40',
             '41', '42', '44', '45', '46', '47', '48', '49', '50', '51', '53',
             '54', '55', '56', '91']

con.execute(f"""
  CREATE OR REPLACE TABLE healthcare_table AS
  SELECT *, '99' AS region FROM read_csv('{path}')
""")
string_to_execute = """UPDATE healthcare_table SET region = CASE fips_st"""
for i in range(len(state_regions_alphabetical)):
    string_to_execute = string_to_execute + f""" WHEN '{fips_code[i]}' THEN '{state_regions_a
string_to_execute = string_to_execute + f"""END"""
con.execute(string_to_execute)

display(con.execute("SELECT * FROM healthcare_table").df().head())

con.execute(f"""
  CREATE OR REPLACE TABLE small_healthcare_table AS
  SELECT fips_st, st_abbrev, phys_wkforc_23, pa_23, rn_23, dent_23,
```

```
    pharm_23, vetin_23, podtrst_23, chiro_23, opto_23, psychol_23,conslrs_23,
    socwk_23, pt_23, ot_23, resp_ther_23, spech_path_23, massg_ther_23, dietn_23,
    medmgr_23, med_secrtrs_23, clin_lab_techs_23, diag_reltd_techs_23, emt_parmdcs_23,
    hlth_sup_techs_23, med_rec_spec_23, med_asst_23, pharm_aide_23, med_transcrp_23,
    vetin_asst_23, phleb_23, oth_hlthcar_23, pers_care_aide_23, nurse_aide_23,
    popn_pums_23, popn_ge60_23, popn_24, region FROM healthcare_table
""")
```

```
             column_name column_type null   key default extra
0                fips_st     VARCHAR  YES  None    None  None
1              st_abbrev     VARCHAR  YES  None    None  None
2         phys_wkforc_23      BIGINT  YES  None    None  None
3            phys_mal_23      BIGINT  YES  None    None  None
4            phys_fem_23      BIGINT  YES  None    None  None
...                  ...         ...  ...   ...     ...   ...
1443   popn_nhpi_ge16_23      BIGINT  YES  None    None  None
1444   popn_aian_ge16_23      BIGINT  YES  None    None  None
1445  popn_2race_ge16_23      BIGINT  YES  None    None  None
1446             popn_24      BIGINT  YES  None    None  None
1447        popn_ge16_24      BIGINT  YES  None    None  None

[1448 rows x 6 columns]
```

	st_abbrev
0	WA
1	NH
2	WY
3	NY
4	PA
5	AR
6	CT
7	MS
8	VT
9	VA
10	WV
11	CO
12	MD
13	MA
14	FL
15	LA

	st_abbrev
16	ME
17	HI
18	NC
19	KS
20	MN
21	NM
22	UT
23	DC
24	IL
25	IN
26	NE
27	US
28	AK
29	CA
30	MI
31	NV
32	TX
33	ID
34	GA
35	WI
36	AL
37	IA
38	MT
39	OH
40	OR
41	AZ
42	DE
43	MO
44	OK
45	KY
46	NJ
47	ND
48	RI
49	SC
50	SD
51	TN

	fips_st	st_abbrev	phys_wkforc_23	phys_mal_23	phys_fem_23	phys_lt30_23	phys_30_39_23
0	01	AL	12442	8745	3697	1347	2871

	fips_st	st_abbrev	phys_wkforc_23	phys_mal_23	phys_fem_23	phys_lt30_23	phys_30_39_23
1	02	AK	2158	1228	930	<NA>	969
2	04	AZ	18253	11945	6308	1195	4341
3	05	AR	6493	4291	2202	<NA>	2045
4	06	CA	115435	66542	48893	6227	32051

```
<_duckdb.DuckDBPyConnection at 0x10ff60270>
```

Exploratory Data Analysis

The below code chunk will create some plots that explore the data

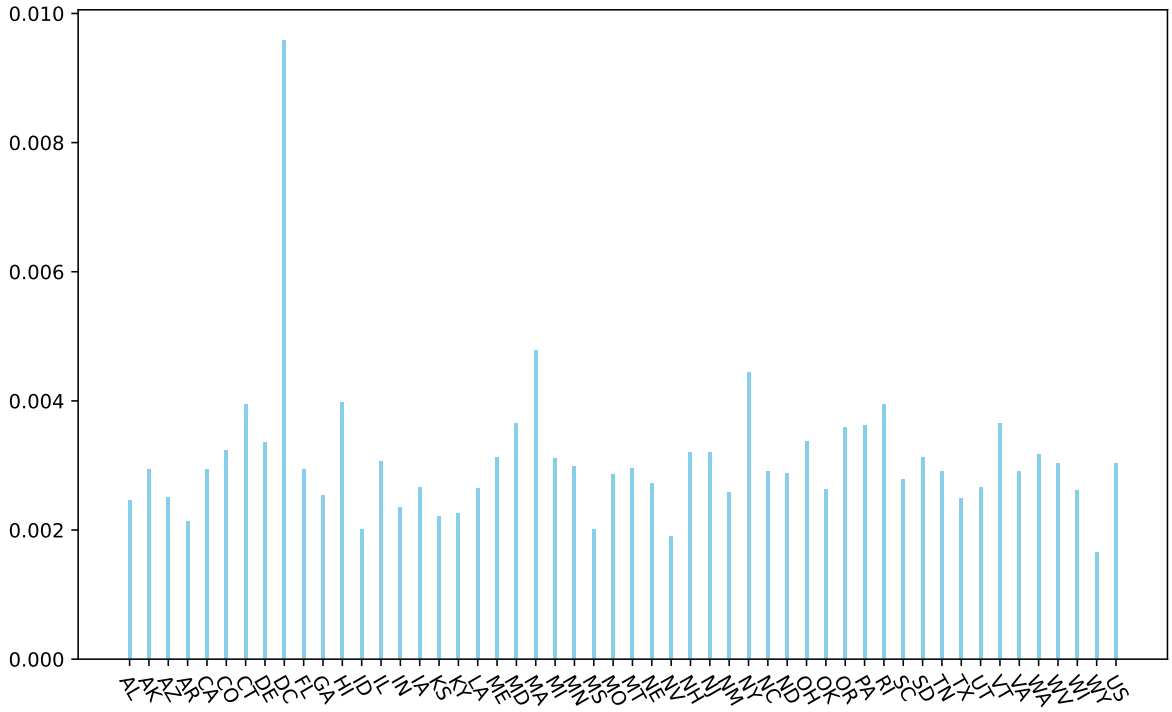
The plots are listed below in order and what they will do

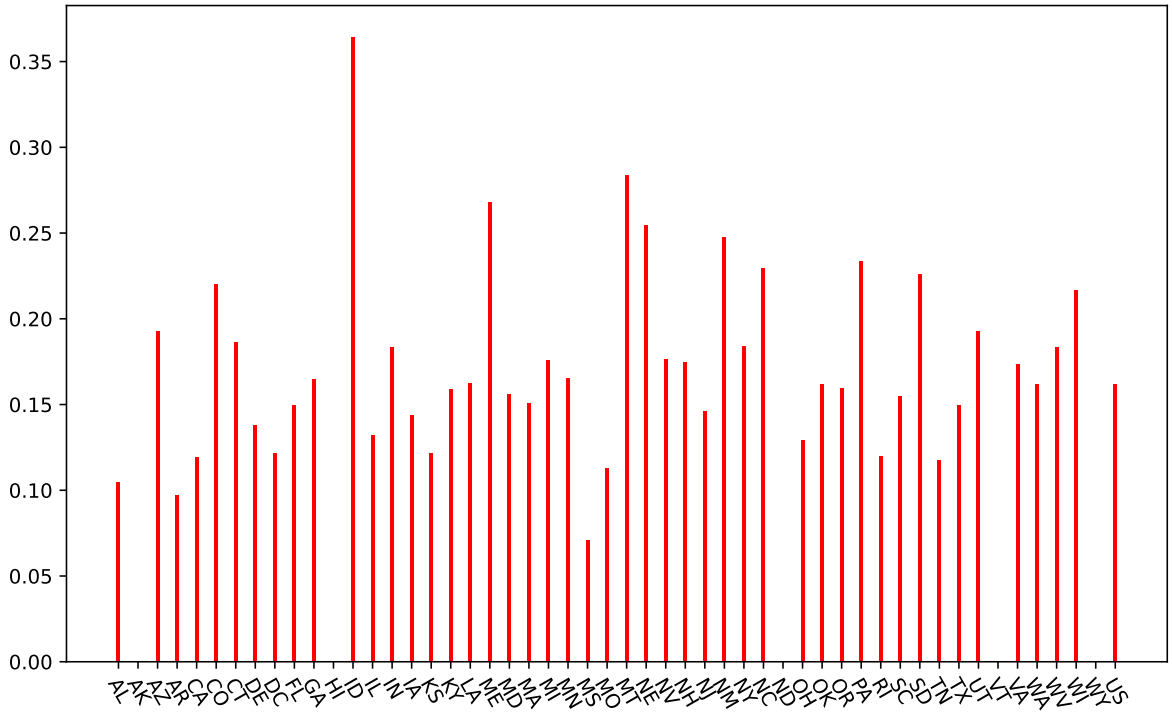
1) The first plot explores the ratio of physicians to the total population of the state
2) The second plot explores the ratio of physicians assistants to the total population of the state

```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 6))
plt.xticks(rotation=300)
plt.bar(smaller_health_pandas_dataframe['st_abbrev'], smaller_health_pandas_dataframe['phys_w
plt.show()

plt.figure(figsize=(10, 6))
plt.xticks(rotation=300)
plt.bar(smaller_health_pandas_dataframe['st_abbrev'], smaller_health_pandas_dataframe['pa_23
plt.show()
```


0.010
0.008
0.006
0.004
0.002
0.000


0.35
0.30
0.25
0.20
0.15
0.10
0.05
0.00

Saving the data

The next code chunk will save the two dataframes as a parquet file for easier storage and future access

```
raw_pandas_dataframe.to_parquet('Data/clean_data/large_2024_health_data.parquet')
smaller_health_pandas_dataframe.to_parquet('Data/clean_data/small_2024_health_data.parquet')
```